Intermediate Code Generator for Control-Flow Statements

if, if-else, while and nested conditionals using Backpatching

Compiler Design Project | Review 1 (25% implementation)

By: Ashutosh Gupta – 24BCE2983, Dhruv Mathur – 24BCE2504, Somya Raj Singh Dalawat – 24BCE3001

Problem Identification

- A one-pass compiler emits code as it parses; a forward jump is generated before its target exists.
- In if (B) S1 else S2, the code for B is emitted before the positions of S1, S2 and the following statement are known.
- Fix 1: symbolic labels + a second resolving pass. Costs an extra pass, label table and memory.
- Fix 2: backpatching. Emit the jump with an empty target, record its index in a list, and fill it in when the target is known.
- Backpatching also suits bottom-up (LR) parsing, where inherited label attributes are not available.

Why Targets Are Unknown

```
if (a < b && c < d) x = 1;
```

```
0: if a < b goto ?   (S?)
1: goto ?            (exit?)
2: if c < d goto ?
3: goto ?
4: x = 1
5: ...
```

- Quads 0-3 are emitted before x = 1 is parsed.
- Their targets (4 and 5) become known only later.
- Backpatching remembers {0..3} and patches them once the targets exist.

Objectives

- Define a grammar for if, if-else, while, nested conditionals and blocks, with &&, ||, ! and relational operators.
- Generate three-address code (quadruples) in a single pass using truelist, falselist and nextlist.
- Implement makelist(), merge(), backpatch() and marker non-terminals M and N.
- Support short-circuit evaluation without materializing boolean values.
- Validate on nested constructs: while inside if, if inside while, and mixed conditions.
- Deliver a documented, testable implementation with traces.

Scope and Source Language

```
S -> id = E ;
  | if ( B ) S
  | if ( B ) S else S
  | while ( B ) S
  | { L }
L -> L S | S
B -> B || B | B && B
  | ! B | ( B ) | E relop E
E -> E + T | E - T | T
     (T, F as usual)
```

- Small C-like language
- Types: integer variables only
- No functions or arrays yet
- Output: quadruple array printed as numbered three-address code

Literature Survey (1/2)

Aho, Lam, Sethi, Ullman: Compilers: Principles, Techniques, and Tools 2006 (2nd ed.)

The standard reference for this topic. Chapter 6 defines truelist, falselist and nextlist, the marker non-terminals M and N, and the full backpatching translation scheme for boolean expressions, if, if-else and while. Our grammar and semantic rules follow this treatment directly.

Aho & Ullman: Principles of Compiler Design 1977

An early formulation of syntax-directed translation for one-pass compilers. It shows how jumps whose targets are not yet known can be left incomplete and fixed up later, which is the idea backpatching refines. Useful for the historical motivation of the problem.

Lewis, Rosenkrantz, Stearns: Attributed Translations (JCSS 9(3)) 1974

Introduced L-attributed definitions, where attributes can be computed in a single left-to-right pass. This explains why inherited labels need top-down parsing and why synthesized lists plus markers are the practical choice for bottom-up (LR) parsers.

Literature Survey (2/2)

Louden: Compiler Construction: Principles and Practice 1997

A practical, implementation-oriented text. It generates code for if and while statements with explicit jump fix-ups in a recursive-descent style. It guided our Python design, where each parsing function returns its attribute lists and patches them at the right moment.

Appel: Modern Compiler Implementation 1998

Represents conditionals as trees with true and false patch lists and compares this with label-based intermediate code. It helps us argue the trade-off between backpatching (no labels, one pass) and label generation (simpler, but needs resolution).

Cooper & Torczon: Engineering a Compiler 2011

Discusses code shapes for control flow, short-circuit evaluation of boolean expressions and how quadruples map to a control-flow graph. It informs our test cases and our planned flow-graph output for the remaining 75% of the project.

Approach Comparison and Gap

Approach	Passes	Pros	Cons
Symbolic labels + resolve pass	2	Simple to implement	Extra pass, label table
Inherited labels (L-attributed)	1	No patching needed	Needs top-down parsing; unused labels
Backpatching	1	Works with LR parsing; no label generation	List bookkeeping

- Gap: sources give SDT rules or fragments; few present a complete, runnable generator with nested traces.
- Contribution: working implementation, boolean short-circuiting, nested examples and a test plan.

Backpatching Concepts

```
emit(s)         append quad, return index
nextquad()      index of next quad
makelist(i)     new list [i]
merge(a, b)     a + b
backpatch(L,t) set target of L to t
```

- B.truelist: jumps taken when B is true
- B.falselist: jumps taken when B is false
- S.nextlist: jumps to the code after S

- Marker M -> epsilon records nextquad for use by later patching.
- Marker N -> epsilon emits goto ? to skip the else part.
- Each jump is patched exactly once, so total work is O(n).

SDT Rules: Boolean Expressions

```
B -> B1 || M B2
   backpatch(B1.false, M.q)
   B.true  = merge(B1.true, B2.true)
   B.false = B2.false

B -> B1 && M B2
   backpatch(B1.true, M.q)
   B.true  = B2.true
   B.false = merge(B1.false, B2.false)
```

```
B -> ! B1
   B.true = B1.false
   B.false = B1.true

B -> E1 relop E2
   emit(if E1 relop
        E2 goto ?)
   true  = [that quad]
   emit(goto ?)
   false = [that quad]
```

SDT Rules: Statements

```
S -> if ( B ) M S1
   backpatch(B.true, M.q)
   S.next = merge(B.false, S1.next)

S -> if ( B ) M1 S1 N else M2 S2
   backpatch(B.true, M1.q); backpatch(B.false, M2.q)
   S.next = merge(S1.next, N.next, S2.next)

S -> while M1 ( B ) M2 S1
   backpatch(S1.next, M1.q); backpatch(B.true, M2.q)
   emit(goto M1.q); S.next = B.false

L -> L1 M S
   backpatch(L1.next, M.q); L.next = S.next
```

Implementation Architecture



- Parser functions return attribute lists: B() returns (truelist, falselist); S() returns nextlist.
- Arithmetic expressions emit temporaries t1, t2, ... through E, T, F.
- Final pass over the program backpatches the last nextlist and emits end.

Worked Trace: if-else

```
if (a < b) x = 1; else x = 2;
```

```
0: if a < b goto 2
1: goto 4
2: x = 1
3: goto 5
4: x = 2
5: end
```

- Quads 0, 1 emitted with '?': true={0}, false={1}
- M1 = 2 patches quad 0
- N emits quad 3 (goto ?), next={3}
- M2 = 4 patches quad 1 (else branch)
- next={3} patched to 5 when end is emitted

Worked Trace: Short-Circuit &&

```
if (a < b && c < d) x = 1;
```

```
0: if a < b goto 2
1: goto 5
2: if c < d goto 4
3: goto 5
4: x = 1
5: end
```

- Left operand false: jump straight to exit (quad 1)
- Left operand true: patched to the right operand (quad 2)
- falselist = {1, 3} merged; both patched to 5
- No boolean value is ever computed

Worked Trace: Nested while + if-else

```
while (a < b) if (c < d) x = 1; else x = 2;
```

```
0: if a < b goto 2
1: goto 8
2: if c < d goto 4
3: goto 6
4: x = 1
5: goto 0
6: x = 2
7: goto 0
8: end
```

- Inner N (quad 5) and the else branch fall to the loop jump
- Quad 7 is the loop back-edge to condition at 0
- Loop falselist {1} exits to 8
- Shows patching across two nesting levels

Implementation Status (25%)

Completed

- Lexical analyzer in Lex for identifiers, constants, relational operators, and logical operators.
- Core backpatching functions implemented in C: makelist(), merge(), backpatch(), and emit().
- Synthesis of truelist, falselist, and nextlist using dynamic linked-list structures.
- Bottom-up jump management using marker non-terminals M (records nextquad) and N (emits skip goto).
- Quadruple generation for if, if-else, while, and nested compound statement blocks.
- Short-circuit evaluation for boolean expressions using &&, ||, and ! without computing boolean flags.
- • Three-address code generation for arithmetic expressions with temporary variable allocation.

Remaining (75%)

- Syntax-error reporting, recovery routines, and symbol table integration.
- Semantic analysis and type-checking rules for expressions and assignments.
- Additional control-flow statements: for loops, do-while loops, switch-case, break, and continue.
- Flow-graph partitioning into basic blocks and visual Control Flow Graph (CFG) generation.
- Quadruple table display and intermediate code optimization (redundant jump elimination).
- Comprehensive test harness evaluating deep nesting and arbitrary boolean compositions.

Implementation – 25%

Open

lexer.l

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "y.tab.h"
%}

%%

[ \t\n\r]+              ;

"if"                    { return IF; }
"else"                  { return ELSE; }
"while"                 { return WHILE; }

"&&"                    { return AND; }
"||"                    { return OR; }
"!"                     { return NOT; }

"<="                    { strcpy(yylval.str, "<="); return RELOP; }
">="                    { strcpy(yylval.str, ">="); return RELOP; }
"=="                    { strcpy(yylval.str, "=="); return RELOP; }
"!="                    { strcpy(yylval.str, "!="); return RELOP; }
"<"                     { strcpy(yylval.str, "<");  return RELOP; }
">"                     { strcpy(yylval.str, ">");  return RELOP; }

"="                     { return '='; }
";"                     { return ';'; }
"("                     { return '('; }
")"                     { return ')'; }
"{"                     { return '{'; }
"}"                     { return '}'; }
"+"                     { return '+'; }
"-"                     { return '-'; }
"*"                     { return '*'; }
"/"                     { return '/'; }

[a-zA-Z_][a-zA-Z0-9_]*  { strcpy(yylval.str, yytext); return ID; }
[0-9]+                  { strcpy(yylval.str, yytext); return NUM; }

.                       { return yytext[0]; }

%%

int yywrap(void) {
    return 1;
}
```

Open

*parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
void yyerror(const char *s);

struct list {
    int quad;
    struct list *next;
};

/* quadruple entry */
struct quad_entry {
    char op[16];
    char arg1[32];
    char arg2[32];
    char result[32];
};

struct quad_entry quads[1000];
int nextquad = 0;
int temp_count = 0;

void emit(const char *op, const char *arg1, const char *arg2, const char *result);
struct list* makelist(int q);
struct list* merge(struct list *p1, struct list *p2);
void backpatch(struct list *p, int target);
void print_quads(void);
char* newtemp(void);
%}

%union {
    int quad;
    struct list *list_ptr;
    char str[64];
    struct {
        struct list *truelist;
        struct list *falselist;
    } b_expr;
    struct {
        struct list *nextlist;
    } s_stmt;
}

%token <str> ID NUM RELOP
%token IF ELSE WHILE
%token AND OR NOT

%type <b_expr> B
%type <s_stmt> S L
%type <str> E
%type <quad> M
%type <list_ptr> N

%left OR
%left AND
```

Implementation – 25%

Open *parser.y

```
%left OR
%left AND
%right NOT
%left '+' '-'
%left '*' '/'

%%

Program:
    L {
        backpatch($1.nextlist, nextquad);
        emit("end", "", "", "");
        print_quads();
    }
    ;

M:
    /* marker */ {
        $$ = nextquad;
    }
    ;

N:
    /* marker */ {
        $$ = makelist(nextquad);
        emit("goto", "", "", "-");
    }
    ;

L:
    L M S {
        backpatch($1.nextlist, $2);
        $$.nextlist = $3.nextlist;
    }
    | S {
        $$.nextlist = $1.nextlist;
    }
    ;

S:
    ID '=' E ';' {
        $$.nextlist = NULL;
        emit("=", $3, "", $1);
    }
    | IF '(' B ')' M S {
        backpatch($3.truelist, $5);
        $$.nextlist = merge($3.falselist, $6.nextlist);
    }
    | IF '(' B ')' M S N ELSE M S {
        backpatch($3.truelist, $5);
        backpatch($3.falselist, $9);
        struct list *temp = merge($6.nextlist, $7);
        $$.nextlist = merge(temp, $10.nextlist);
    }
    | WHILE M '(' B ')' M S {
        backpatch($7.nextlist, $2);
        backpatch($4.truelist, $6);
        char target[16];
```

Open *parser.y

```
        sprintf(target, "%d", $2);
        emit("goto", "", "", target);
        $$.nextlist = $4.falselist;
    }
    | '{' L '}' {
        $$.nextlist = $2.nextlist;
    }
    ;

B:
    B OR M B {
        backpatch($1.falselist, $3);
        $$.truelist = merge($1.truelist, $4.truelist);
        $$.falselist = $4.falselist;
    }
    | B AND M B {
        backpatch($1.truelist, $3);
        $$.truelist = $4.truelist;
        $$.falselist = merge($1.falselist, $4.falselist);
    }
    | NOT B {
        $$.truelist = $2.falselist;
        $$.falselist = $2.truelist;
    }
    | '(' B ')' {
        $$ = $2;
    }
    | E RELOP E {
        $$.truelist = makelist(nextquad);
        char cond[256];
        sprintf(cond, "%s %s %s", $1, $2, $3);
        emit("if", cond, "goto", "-");
        $$.falselist = makelist(nextquad);
        emit("goto", "", "", "-");
    }
    ;

E:
    E '+' E {
        char *t = newtemp();
        emit("+", $1, $3, t);
        strcpy($$, t);
    }
    | E '-' E {
        char *t = newtemp();
        emit("-", $1, $3, t);
        strcpy($$, t);
    }
    | E '*' E {
        char *t = newtemp();
        emit("*", $1, $3, t);
        strcpy($$, t);
    }
    | E '/' E {
        char *t = newtemp();
        emit("/", $1, $3, t);
        strcpy($$, t);
    }
```

Implementation – 25%

Open *parser.y

```
        strcpy($$, t);
    }
    | '(' E ')' {
        strcpy($$, $2);
    }
    | ID {
        strcpy($$, $1);
    }
    | NUM {
        strcpy($$, $1);
    }
    ;

%%

char* newtemp(void) {
    static char buf[16];
    sprintf(buf, "t%d", ++temp_count);
    return buf;
}

void emit(const char *op, const char *arg1, const char *arg2, const char *result) {
    strcpy(quads[nextquad].op, op);
    strcpy(quads[nextquad].arg1, arg1);
    strcpy(quads[nextquad].arg2, arg2);
    strcpy(quads[nextquad].result, result);
    nextquad++;
}

struct list* makelist(int q) {
    struct list *node = (struct list*)malloc(sizeof(struct list));
    node->quad = q;
    node->next = NULL;
    return node;
}

struct list* merge(struct list *p1, struct list *p2) {
    if (!p1) return p2;
    if (!p2) return p1;
    struct list *curr = p1;
    while (curr->next != NULL) {
        curr = curr->next;
    }
    curr->next = p2;
    return p1;
}

void backpatch(struct list *p, int target) {
    char target_str[16];
    sprintf(target_str, "%d", target);
    struct list *curr = p;
    while (curr != NULL) {
        strcpy(quads[curr->quad].result, target_str);
        curr = curr->next;
    }
}

void print_quads(void) {
    for (int i = 0; i < nextquad; i++) {
        if (strcmp(quads[i].op, "=") == 0) {
            printf("%2d: %s = %s\n", i, quads[i].result, quads[i].arg1);
        } else if (strcmp(quads[i].op, "if") == 0) {
            printf("%2d: if %s goto %s\n", i, quads[i].arg1, quads[i].result);
        } else if (strcmp(quads[i].op, "goto") == 0) {
            printf("%2d: goto %s\n", i, quads[i].result);
        } else if (strcmp(quads[i].op, "end") == 0) {
            printf("%2d: end\n", i);
        } else {
            printf("%2d: %s = %s %s %s\n", i, quads[i].result, quads[i].arg1, quads[i].op, quads[i].arg2);
        }
    }
}

void yyerror(const char *s) {
    fprintf(stderr, "Syntax error: %s\n", s);
}

int main(void) {
    yyparse();
    return 0;
}
```

Output:

```
ashutosh@LAPTOP-QJMHB5V\
ashutosh@LAPTOP-QJMHB5VK:~$ lex lexer.l
ashutosh@LAPTOP-QJMHB5VK:~$ yacc -d parser.y
parser.y: warning: 1 reduce/reduce conflict [-Wconflicts-rr]
parser.y: note: rerun with option '-Wcounterexamples' to generate conflict counterexamples
ashutosh@LAPTOP-QJMHB5VK:~$ gcc y.tab.c lex.yy.c -o icg
ashutosh@LAPTOP-QJMHB5VK:~$ ./icg < input.txt
 0: if a < b goto 2
 1: goto 8
 2: if c < d goto 4
 3: goto 6
 4: x = 1
 5: goto 0
 6: x = 2
 7: goto 0
 8: end
```

Input :

while (a < b) if (c < d) x = 1; else x = 2;

References

- Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed., 2006.
- Aho & Ullman. Principles of Compiler Design, 1977.
- Lewis, Rosenkrantz, Stearns. Attributed Translations. JCSS 9(3), 1974.
- Louden. Compiler Construction: Principles and Practice, 1997.
- Appel. Modern Compiler Implementation, 1998.
- Cooper & Torczon. Engineering a Compiler, 2nd ed., 2011.